



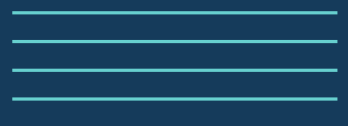
RELATÓRIO TÉCNICO

ROTA VITAL

Análise de desempenho entre
processamento sequencial e paralelo

Operação analisada: cálculo de estatísticas por hospital
Cargas de teste: 100.000 e 1.000.000 de requisições
Configurações: 1, 2, 4 e 8 threads

2026



Sumário

Sumário	2
Resumo executivo	2
1. Introdução	3
2. Operação escolhida e justificativa	3
3. Complexidade computacional	3
4. Identificação do gargalo	3
5. Possibilidade de particionamento	4
6. Metodologia do experimento	4
7. Resultados obtidos	5
8. Análise do speedup	6
9. Análise da eficiência	7
10. Discussão dos resultados	8
10.1 Influência do tamanho da entrada	8
10.2 Ganho real e limite ideal	8
10.3 Interpretação da configuração com oito threads	8
10.4 A complexidade Big-O mudou?	8
10.5 Concorrência x paralelismo: comparação com a Mesa DJ	9
10.6 Quando oito threads não forem suficientes	9
11. Limitações e cuidados de medição	10
12. Conclusão	11
Apêndice A - Fórmulas e cálculos principais	11

Resumo executivo

Este relatório compara o desempenho do cálculo de estatísticas por hospital no sistema **Rota Vital** em uma implementação sequencial e em configurações paralelas com 2, 4 e 8 threads. Foram analisadas cargas de 100.000 e 1.000.000 de requisições, utilizando tempo de execução, speedup e eficiência como métricas.

Principal resultado

Na carga de 1.000.000 de requisições, o tempo caiu de **249,80 ms** com uma thread para **63,40 ms** com oito threads. Isso corresponde a um speedup de **3,94x** e redução aproximada de **74,6%** no tempo de execução.

1. Introdução

O objetivo deste trabalho é analisar o impacto do processamento paralelo sobre uma operação computacional do sistema **Rota Vital**. A comparação foi realizada entre uma execução sequencial, com uma única thread, e execuções paralelas utilizando 2, 4 e 8 threads.

Foram empregadas duas cargas distintas: 100.000 e 1.000.000 de requisições de atendimento hospitalar. Além dos tempos médios, foram calculados o speedup e a eficiência, permitindo avaliar tanto a redução do tempo quanto o aproveitamento dos recursos de processamento.

2. Operação escolhida e justificativa

A operação escolhida foi o **cálculo de estatísticas por hospital**, implementado no método **calcularSequencial**. O método recebe uma lista de requisições e, para cada elemento, atualiza um acumulador do tipo **EstatisticaHospital**, indexado pelo identificador do hospital em um **HashMap**.

Durante o processamento, são somados os tempos de atendimento e incrementado o total de requisições de cada hospital. Ao final, os valores acumulados permitem calcular a média de atendimento por hospital.

A escolha é representativa porque combina uma operação simples com grande repetição. Não existe dependência direta entre as requisições, além da agregação final por hospital, o que favorece a divisão do trabalho entre threads.

Por que esta operação é adequada?

O trabalho é repetitivo, executado integralmente em memória e pode ser dividido em blocos independentes. Essas características permitem observar com clareza os ganhos e os custos do paralelismo.

3. Complexidade computacional

Na versão sequencial, a lista é percorrida uma única vez. Para cada requisição ocorre uma busca ou inserção no **HashMap**, seguida da atualização dos valores acumulados.

Como a busca e a inserção em um **HashMap** apresentam custo médio amortizado $O(1)$, o custo total do algoritmo é:

$$T(n) = O(n)$$

Nessa expressão, **n** representa o número de requisições. Não existem laços aninhados nem ordenação, portanto o tempo tende a crescer aproximadamente de forma proporcional ao tamanho da entrada.

4. Identificação do gargalo

O processamento analisado é predominantemente limitado por CPU e acesso à memória, e não por banco de dados ou rede. As requisições são geradas previamente em memória pela classe **GeradorRequisicoes**, antes do início da medição.

A medição com **System.nanoTime()** cobre apenas o cálculo das estatísticas. Durante essa etapa, cada elemento exige operações aritméticas simples, consultas ao mapa e atualizações de acumuladores. Não há espera por serviços externos.

Esse perfil favorece o uso de múltiplos núcleos. Se o gargalo estivesse em uma fonte externa compartilhada, como rede ou banco de dados, adicionar threads poderia apenas aumentar a quantidade de tarefas aguardando a mesma resposta.

5. Possibilidade de particionamento

As requisições podem ser processadas em qualquer ordem, e o cálculo de um hospital não depende do cálculo de outro. Soma e contagem são operações associativas: resultados parciais podem ser combinados sem alterar o resultado final.

No projeto, a lista é dividida pelo método **dividirLista**, utilizando **subList**. Cada thread processa uma parcela e gera estatísticas locais. Ao final, ocorre a junção dos mapas parciais.

Classificação

O problema se aproxima de um caso embarrassingly parallel: há pouca comunicação entre threads durante o processamento e a sincronização principal acontece somente na combinação final dos resultados.

6. Metodologia do experimento

Foram avaliadas quatro configurações de execução para cada volume de dados: 1, 2, 4 e 8 threads. A configuração de uma thread funciona como referência sequencial. Os demais tempos são comparados a essa referência para calcular speedup e eficiência.

Elemento	Configuração
Volumes processados	100.000 e 1.000.000 de requisições
Paralelismo	1, 2, 4 e 8 threads
Temporização	System.nanoTime()
Métricas	Tempo médio, speedup e eficiência
Escopo medido	Somente o processamento das estatísticas

7. Resultados obtidos

A tabela a seguir apresenta os tempos médios registrados. Para 100.000 requisições, a execução com duas threads foi mais lenta que a sequencial, enquanto quatro threads apresentaram desempenho praticamente equivalente. O melhor tempo foi obtido com oito threads.

Volume	1 thread	2 threads	4 threads	8 threads
100.000	7,80 ms	10,40 ms	8,00 ms	4,00 ms
1.000.000	249,80 ms	148,80 ms	88,40 ms	63,40 ms

Na carga de 1.000.000 de requisições, o aumento do número de threads produziu redução consistente do tempo: 249,80 ms, 148,80 ms, 88,40 ms e 63,40 ms, respectivamente.

Tempo de execução por configuração

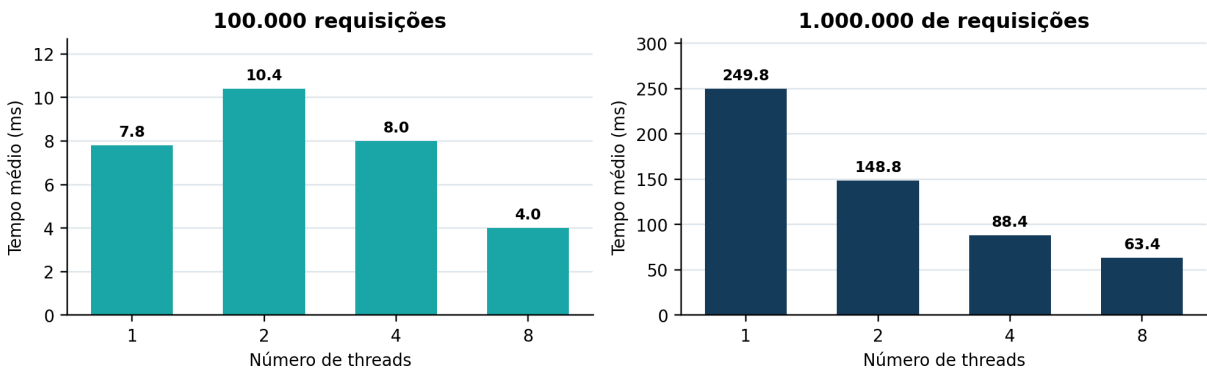


Figura 1 - Tempo de execução por configuração. Fonte: elaborado pelos autores (2026).

8. Análise do speedup

O speedup indica quantas vezes a execução paralela foi mais rápida que a referência sequencial. Seu cálculo é dado por:

$$S(p) = T(1) / T(p)$$

em que $T(1)$ é o tempo com uma thread e $T(p)$ é o tempo utilizando p threads.

Volume	1 thread	2 threads	4 threads	8 threads
100.000	1,00x	0,75x	0,98x	1,95x
1.000.000	1,00x	1,68x	2,83x	3,94x

Na carga menor, speedup inferior a 1 com duas threads significa que o paralelismo foi mais lento. Com quatro threads, o desempenho ficou praticamente empatado com o sequencial. O ganho apareceu de forma mais clara com oito threads, chegando a 1,95x.

Para 1.000.000 de requisições, os ganhos foram progressivos: 1,68x, 2,83x e 3,94x. Isso confirma que uma carga maior consegue amortizar melhor o custo adicional da execução paralela.

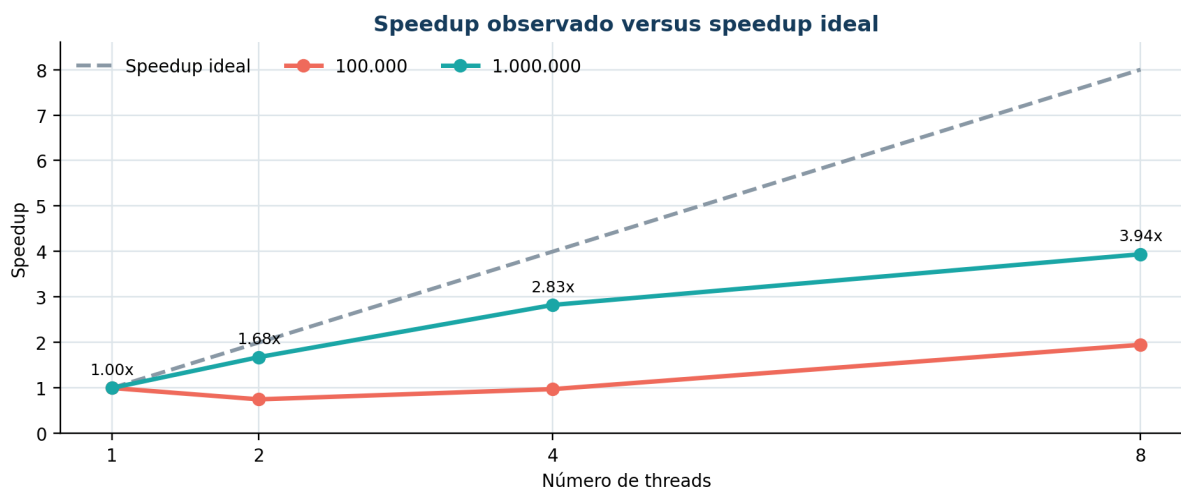


Figura 2 - Speedup observado em comparação ao speedup ideal. Fonte: elaborado pelos autores (2026).

9. Análise da eficiência

A eficiência mede o aproveitamento médio dos recursos paralelos. Ela relaciona o speedup à quantidade de threads utilizada:

$$E(p) = S(p) / p$$

O resultado pode ser expresso em porcentagem multiplicando a razão por 100.

Volume	1 thread	2 threads	4 threads	8 threads
100.000	100,0%	37,5%	24,4%	24,4%
1.000.000	100,0%	83,9%	70,6%	49,3%

Com 100.000 registros, a eficiência foi baixa nas configurações paralelas porque o trabalho útil é pequeno em relação ao custo de gerenciamento das threads. Com 1.000.000 de registros, duas threads atingiram 83,9% de eficiência e quatro alcançaram 70,6%.

A queda para 49,3% com oito threads não significa ausência de ganho. Essa configuração ainda apresentou o menor tempo absoluto, mas cada thread adicional ofereceu benefício proporcionalmente menor.

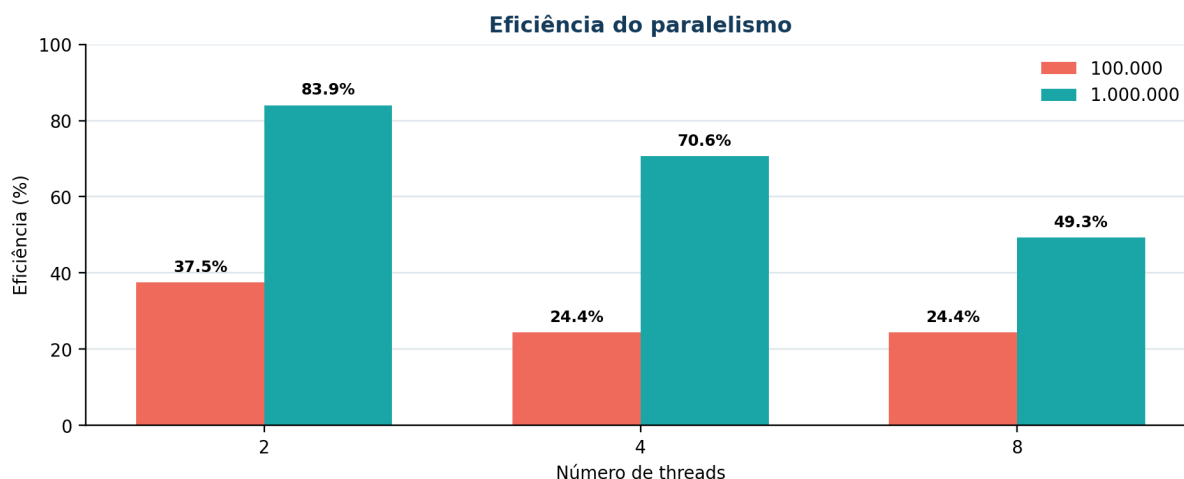


Figura 3 - Eficiência das configurações paralelas. Fonte: elaborado pelos autores (2026).

10. Discussão dos resultados

10.1 Influência do tamanho da entrada

Com 100.000 requisições, o processamento sequencial já termina em apenas 7,80 ms. Nessa escala, operações administrativas do paralelismo, como divisão da lista, criação ou reutilização de tarefas, agendamento, sincronização e junção dos mapas, representam parcela significativa do tempo total.

Com 1.000.000 de requisições, existe trabalho suficiente para distribuir. O custo fixo do paralelismo passa a representar uma fração menor da execução, permitindo que os núcleos trabalhem simultaneamente com benefício mais evidente.

10.2 Ganho real e limite ideal

O speedup ideal seria igual ao número de threads: 2x, 4x e 8x. Os valores observados ficaram abaixo desse limite. Com oito threads e 1.000.000 de registros, o speedup foi 3,94x, praticamente metade do máximo teórico.

Essa diferença é esperada. Nem todas as etapas são paralelizáveis, e ainda existem custos de comunicação, memória, escalonamento e combinação dos resultados. Conforme novas threads são adicionadas, esses fatores reduzem o ganho marginal.

10.3 Interpretação da configuração com oito threads

Oito threads produziram o menor tempo nos dois volumes, mas a eficiência ficou em 24,4% para 100.000 registros e 49,3% para 1.000.000. Portanto, essa configuração priorizou o menor tempo de resposta, não o melhor aproveitamento por thread.

Leitura prática

Para cargas pequenas, a versão sequencial pode ser suficiente e mais previsível. Para cargas grandes, a versão paralela oferece ganhos claros. Entre as configurações testadas, oito threads entregaram o menor tempo; duas ou quatro threads podem ser preferíveis quando eficiência e consumo de recursos também forem importantes.

10.4 A complexidade Big-O mudou?

Não. A versão sequencial continua com complexidade $O(n)$, pois cada uma das n requisições é processada uma vez. Na versão paralela com p threads, a lista é dividida em blocos de aproximadamente n/p elementos. Assim, cada thread executa cerca de $O(n/p)$ operações.

Considerando também a combinação dos resultados parciais, uma forma mais completa de representar o tempo é $O(n/p + p.h)$, em que h é a quantidade de hospitais presentes nos mapas parciais. O primeiro termo representa o processamento dos blocos e o segundo, a junção dos resultados gerados pelas p threads.

Quando p é fixo e h é pequeno em relação a n , a expressão simplifica assintoticamente para $O(n)$. Portanto, o paralelismo reduz o tempo real por meio da execução simultânea, mas não altera a ordem de crescimento do problema. O trabalho total ainda cresce linearmente conforme aumenta o número de requisições.

Resposta direta

Sequencial: $O(n)$. Paralelo: aproximadamente $O(n/p)$ por thread, mais o custo da junção. Com quantidade fixa de threads, a complexidade total permanece $O(n)$.

10.5 Concorrência x paralelismo: comparação com a Mesa DJ

Concorrência e paralelismo são conceitos relacionados, mas não equivalentes. A concorrência descreve várias tarefas em progresso no mesmo intervalo de tempo; elas podem apenas se alternar em um único núcleo. O paralelismo exige que duas ou mais tarefas sejam realmente executadas ao mesmo tempo, normalmente em núcleos diferentes.

Aspecto	Concorrência	Paralelismo
Ideia principal	Várias tarefas avançam por alternância.	Várias tarefas são executadas simultaneamente.
Mesa DJ	Um DJ administra pedidos, faixas e dois decks, alternando a atenção entre eles.	Dois ou mais DJs ou unidades independentes executam trabalhos ao mesmo tempo.
Rota Vital	As threads podem ficar prontas e ser intercaladas pelo sistema operacional.	Partes da lista são processadas simultaneamente em diferentes núcleos da CPU.
Requisito	Pode ocorrer mesmo em uma CPU com apenas um núcleo.	Depende de recursos físicos capazes de executar tarefas ao mesmo tempo.

Na analogia da Mesa DJ, um único DJ pode manter várias atividades concorrentes, escolhendo a próxima música, atendendo pedidos e preparando outra faixa por alternância. Há paralelismo quando atividades independentes são realizadas simultaneamente por mais de um DJ, mesa ou unidade de processamento. Da mesma forma, criar várias threads no Rota Vital garante concorrência, mas o paralelismo real depende de essas threads serem executadas simultaneamente por diferentes núcleos.

10.6 Quando oito threads não forem suficientes

Adicionar threads indefinidamente não resolve o problema. Depois que todos os núcleos estão ocupados, novas threads aumentam disputa por CPU, memória e sincronização. Se a carga ultrapassar a capacidade de uma única máquina, a evolução adequada é combinar otimização local com **escalabilidade horizontal**.

Etapa	Evolução arquitetural	Objetivo
1. Otimização local	Pool de threads ajustado aos núcleos, processamento em lotes e redução do custo de junção.	Extrair melhor desempenho da máquina atual.
2. Escala vertical	Mais núcleos, memória e melhor capacidade de processamento.	Aumentar o limite de uma única instância.
3. Escala horizontal	Fila de tarefas e múltiplos workers em máquinas ou contêineres diferentes.	Distribuir a carga entre várias instâncias.
4. Agregação distribuída	Cada worker produz estatísticas parciais; uma etapa de redução combina os mapas.	Preservar o mesmo resultado em um modelo semelhante a MapReduce.
5. Controle operacional	Balanceamento, backpressure, retentativas, idempotência e monitoramento.	Manter estabilidade, rastreabilidade e tolerância a falhas.

Uma arquitetura possível envia lotes a uma fila, distribui-os entre workers e encaminha resultados parciais a um agregador. Como soma e contagem são associativas, a lógica de divisão e junção pode ser ampliada para vários processos ou máquinas.

Evolução recomendada

Quando oito threads não bastarem, o próximo passo não é apenas criar mais threads. A arquitetura deve evoluir de paralelismo local para processamento distribuído, com filas, workers escaláveis e agregação final dos resultados.

11. Limitações e cuidados de medição

Os números devem ser interpretados como resultados do ambiente específico em que o teste foi executado. O relatório não recebeu informações sobre modelo de processador, número de núcleos físicos, sistema operacional, versão da JVM ou quantidade exata de repetições.

Em aplicações Java, aquecimento da JVM, compilação JIT, coleta de lixo e outras tarefas do sistema podem influenciar medições curtas. Isso é especialmente relevante no caso de 100.000 registros, em que diferenças de poucos milissegundos alteram bastante o speedup calculado.

Para aumentar a confiabilidade em trabalhos futuros, recomenda-se executar rodadas de aquecimento, repetir cada cenário várias vezes, registrar mediana e dispersão, manter o mesmo ambiente e documentar o hardware. Ferramentas específicas de microbenchmark, como o JMH, também podem reduzir erros de medição.

12. Conclusão

Os testes demonstram que o processamento paralelo pode reduzir de forma significativa o tempo do cálculo de estatísticas do Rota Vital, principalmente quando a entrada contém volume suficiente para compensar o custo de gerenciamento das threads.

Para 100.000 requisições, duas threads foram mais lentas que a execução sequencial e quatro threads ficaram praticamente empatadas. Com oito threads, o tempo caiu de 7,80 ms para 4,00 ms, resultando em speedup de 1,95x.

Para 1.000.000 de requisições, o benefício foi mais consistente. O tempo caiu de 249,80 ms com uma thread para 63,40 ms com oito threads, correspondendo a speedup de 3,94x e redução aproximada de 74,6% no tempo.

A eficiência diminuiu conforme mais threads foram adicionadas, evidenciando que o desempenho não cresce de forma linear. A complexidade assintótica também não muda: permanece $O(n)$ com quantidade fixa de threads, embora o tempo real diminua pela divisão do trabalho.

A independência entre as requisições, a divisão em blocos e a junção associativa dos resultados confirmam que a operação é uma boa candidata à paralelização. Caso a capacidade de uma máquina seja superada, essa mesma característica permite evoluir para uma arquitetura distribuída com filas, múltiplos workers e agregação final.

Conclusão final

O paralelismo apresentou melhor relação custo-benefício na maior carga. O experimento confirma tanto o potencial de aceleração quanto os custos inevitáveis da execução com múltiplas threads.

Apêndice A - Fórmulas e cálculos principais

Métrica	Fórmula	Exemplo: 1.000.000 com 8 threads
Speedup	$S = T1 / Tp$	$249,80 / 63,40 = 3,94x$
Eficiência	$E = S / p$	$3,94 / 8 = 49,3\%$
Redução de tempo	$(T1 - Tp) / T1$	$(249,80 - 63,40) / 249,80 = 74,6\%$

Fonte dos dados: resultados experimentais fornecidos pela equipe do projeto Rota Vital. Tabelas, cálculos e gráficos elaborados pelos autores (2026).